Industr.IA

E-book

Panorama de IA Aplicada:

vocabulário, arco histórico e taxonomia de paradigmas e tarefas



Curso base de IA | Unidade M0

Prof.: Arthur Nascimento

SUMÁRIO

0.1. Vocabulário fundacional 3
0.2. Arco histórico e arquitetural 10
0.3. Taxonomia de paradigmas e tarefas 16
Conclusão 22

Unidade M0

0.1. Vocabulário fundacional

Esta seção pertence à camada conceitual e tem por objetivo que você reconheça e use com precisão o conjunto de termos sobre o qual todo o curso se apoia: modelo, treinamento, inferência, parâmetro, generalização, overfitting, a separação dos dados em conjuntos e as métricas básicas de avaliação. Nenhuma matemática avançada é necessária aqui; o que importa é fixar o significado operacional de cada palavra.

Comece pela palavra mais usada e menos definida do campo. Um modelo, em aprendizado de máquina, é uma função com valores ajustáveis que recebe uma entrada e produz uma saída. Um classificador de imagens recebe pixels e devolve um rótulo; um modelo de preço recebe atributos de um imóvel e devolve um número. O que torna um modelo útil não é a forma da função em si, mas o fato de seus valores internos serem aprendidos a partir de exemplos, em vez de escritos à mão por um programador. Esses valores internos são os parâmetros do modelo, e a quantidade deles varia de poucas dezenas, numa regressão linear, a centenas de bilhões, num modelo de linguagem grande.

PARA FIXAR

Para fixar a intuição, imagine que o seu trabalho é desenhar uma reta que separe pontos azuis de pontos vermelhos espalhados num plano. Cada ponto é um exemplo, a posição da reta é o que se ajusta, e a regra é: se a reta atual erra um ponto, mova-a um pouco na direção que tende a acertar. Esse é o aprendizado em sua forma mais simples. Modelos reais trocam a reta por funções com muito mais parâmetros e o plano por espaços com muitas dimensões, mas a mecânica de fundo é a mesma.

A noção de que aprender é “ajustar uma função até que ela acerte os exemplos disponíveis” é o que une regressão linear, árvores de decisão e modelos de linguagem grandes na mesma família conceitual, ainda que as funções concretas sejam muito diferentes.

Treinamento é o processo de ajustar os parâmetros para que as saídas do modelo se aproximem das saídas desejadas nos exemplos disponíveis. Em cada passagem, o treinamento mede o erro do modelo, calcula em que direção cada parâmetro deveria mudar para reduzir esse erro e aplica o ajuste; o laço se repete muitas vezes. Em forma de pseudocódigo, isso se parece com:

```
# durante o treinamento
for exemplo, resposta in treino:
    previsao = modelo(exemplo)
    erro = comparar(previsao, resposta)
    ajustar(modelo, erro)
```

O coração desse laço está em “ajustar”. Como o treinamento sabe em que direção mover cada parâmetro? A intuição é simples: para cada parâmetro, o treinamento pergunta “se eu aumentasse um pouquinho este valor, o erro subiria ou desceria?” e mexe na direção que faz o erro descer. Essa direção de descida, calculada para todos os parâmetros ao mesmo tempo, é o que se chama de **gradiente**, e o algoritmo padrão de ajuste recebe o nome de descida do gradiente (gradient descent). Quanto se anda em cada passo é determinado pela taxa de aprendizado, um dos hiperparâmetros mais sensíveis do treinamento: passo grande demais faz o ajuste oscilar em torno do alvo sem chegar, passo pequeno demais demora a convergir e pode parar antes da hora num ponto que não é o melhor possível.

Uma palavra sobre o ritmo do treinamento. Passar pelo conjunto de treino inteiro uma vez é uma época; o treinamento costuma envolver dezenas, centenas ou milhares de épocas, dependendo do problema e do tamanho do modelo. Mostrar todos os exemplos de uma vez antes de cada ajuste é caro e, muitas vezes, nem cabe na memória, por isso o conjunto de treino é dividido em lotes (batches) menores, e cada ajuste se baseia em um lote por vez. Cada ajuste sobre um lote é uma **iteração**.

> ⚠ ATENÇÃO
>
> Esse vocabulário (época, batch, iteração) aparece em qualquer biblioteca de treinamento e sua distinção importa quando se interpreta logs de execução, métricas reportadas em tempo real e curvas de aprendizado.

Inferência é o uso do modelo já treinado: você apresenta uma entrada nova e lê a saída, sem alterar mais nenhum parâmetro. Em pseudocódigo, o laço some; cada chamada é independente:

```
# em produção, com o modelo já treinado
previsao = modelo(entrada_nova)
```

A distinção entre treinamento e inferência é prática, não apenas terminológica. Treinamento costuma ser caro, demorado e feito poucas vezes; inferência é o que acontece em produção, repetidamente, e é onde moram boa parte das decisões de custo e latência que os módulos seguintes vão tratar.

Nem todo valor que governa o comportamento de um modelo é aprendido no treinamento. Alguns são fixados antes, pela pessoa que conduz o experimento, e moldam como o aprendizado ocorre.

GLOSSÁRIO

hiperparâmetro

Valor de configuração definido antes do treinamento e não ajustado por ele. Exemplos: a taxa de aprendizado, a profundidade máxima de uma árvore, o número de camadas de uma rede. Diferem dos parâmetros, que o treinamento ajusta. A escolha de hiperparâmetros é decidida por experimentação, normalmente sobre o conjunto de validação.

Para fixar a diferença, o quadro a seguir põe parâmetro e hiperparâmetro lado a lado.

Quadro 1. Parâmetro versus hiperparâmetro.

Aspecto	Parâmetro	Hiperparâmetro
Quem ajusta	O treinamento, automaticamente	A pessoa que conduz o experimento
Quando	Durante o treino	Antes do treino, e mantido fixo durante ele
Exemplos	Pesos de uma rede, coeficientes de uma regressão	Taxa de aprendizado, profundidade máxima de uma árvore, número de camadas de uma rede

"Para cego ver":

Imagem de uma tabela intitulada "Quadro 1. Parâmetro versus hiperparâmetro". A tabela possui três colunas: Aspecto, Parâmetro e Hiperparâmetro, organizadas em fundo cinza-claro.

Na primeira linha, referente a "Quem ajusta", consta que o parâmetro é ajustado automaticamente durante o treinamento, enquanto o hiperparâmetro é definido pela pessoa que conduz o experimento.

Na segunda linha, referente a "Quando", informa-se que o parâmetro é ajustado durante o treino, e o hiperparâmetro é definido antes do treinamento e permanece fixo durante sua execução.

Na terceira linha, de "Exemplos", o parâmetro é exemplificado por pesos de uma rede neural e coeficientes de uma regressão. Já os hiperparâmetros incluem taxa de aprendizado, profundidade máxima de uma árvore de decisão e número de camadas de uma rede neural.

O objetivo de treinar um modelo não é acertar os exemplos que ele já viu, e sim acertar exemplos novos. Essa capacidade de ir bem em dados que não estavam no treinamento chama-se generalização, e é o único desempenho que interessa na prática. O principal obstáculo a ela é ajustar o modelo em excesso ao conjunto de treinamento, ao ponto de memorizar particularidades e ruído que não se repetem em dados novos.

GLOSSÁRIO

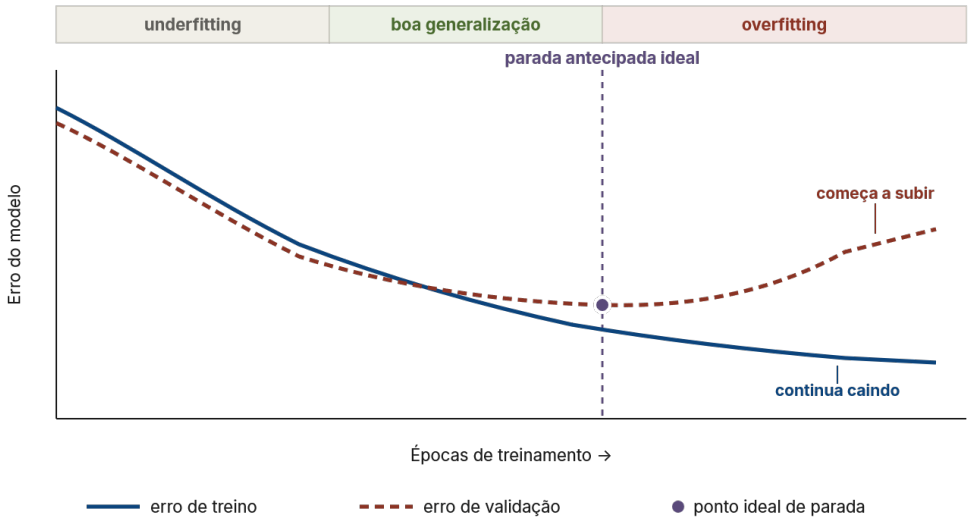
overfitting

Situação em que o modelo vai muito bem nos dados de treinamento e mal em dados novos, porque ajustou-se a ruído e a particularidades irrelevantes. O extremo oposto é o underfitting, quando o modelo é simples demais para capturar o padrão. Técnicas de regularização, que penalizam complexidade, ajudam a conter o overfitting.

A diferença entre underfitting e overfitting fica visível quando se acompanha o erro em treino e em validação ao longo das épocas. No começo do treinamento, ambos caem juntos: o modelo está aprendendo padrões reais, úteis tanto nos exemplos vistos quanto em exemplos novos.

A partir de certo ponto, o erro de treino continua caindo, mas o de validação estabiliza e em seguida começa a subir; é o momento em que o modelo passou a memorizar particularidades do conjunto de treino que não ajudam em dados novos. O ponto em que as duas curvas começam a se separar é o sinal mais claro de overfitting e dá a base para uma técnica simples de mitigação, a parada antecipada (early stopping), que interrompe o treinamento antes que a curva de validação piore. A figura a seguir mostra essa dinâmica.

Figura 1 - Curvas de erro em treino e em validação ao longo das épocas.



Fonte: autor.

"Para cego ver":

Imagem de um gráfico intitulado "Figura 1. Curvas de erro em treino e em validação ao lon o das épocas". O gráfico ilustra a evolução do erro de um modelo de aprendizado de máquina durante o treinamento.

O eixo vertical representa o erro do modelo e o eixo horizontal representa as épocas de treinamento. Duas curvas são apresentadas: uma linha azul contínua indica o erro de treino e uma linha vermelha tracejada representa o erro de validação.

No início do treinamento, ambas as curvas apresentam valores altos e diminuem gradualmente. A região inicial do gráfico é identificada como underfitting. Em seguida, há uma faixa central denominada boa generalização, onde os erros de treino e validação são baixos e próximos entre si.

Mais à direita, uma linha vertical pontilhada destaca o ponto ideal de parada antecipada, indicado por um ponto roxo. A partir desse ponto, a curva de validação deixa de diminuir e começa a subir, sinalizando aumento do erro de validação. Essa região é identificada como overfitting. Enquanto isso, a curva de treino continua diminuindo lentamente.

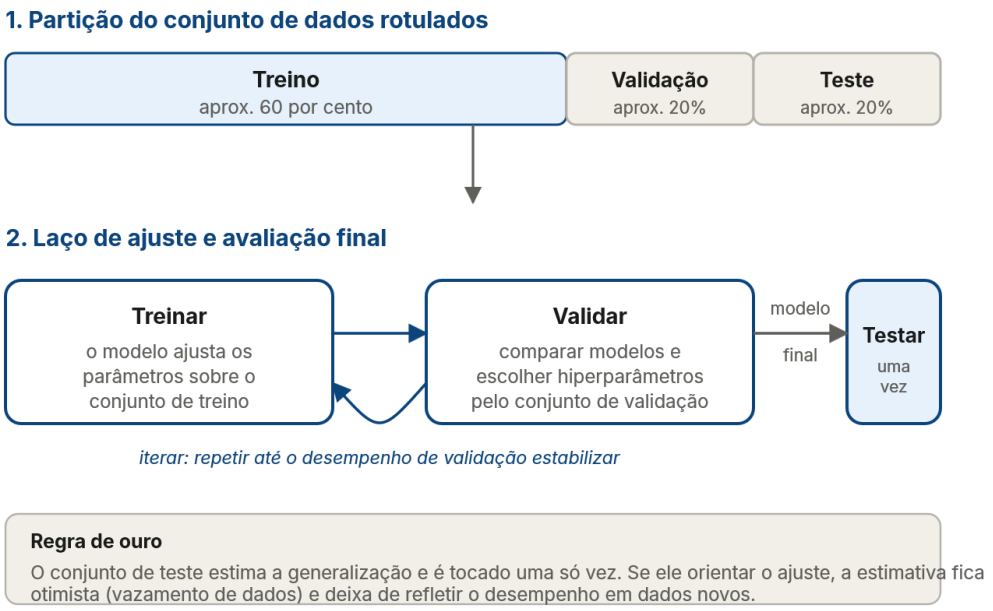
Anotações no gráfico indicam que a curva vermelha de validação "começa a subir" e a curva azul de treino "continua caindo". Na parte inferior, uma legenda associa a linha azul ao erro de treino, a linha vermelha tracejada ao erro de validação e o ponto roxo ao ponto ideal de parada.

Ao final da imagem, abaixo do gráfico, aparece a indicação "Fonte: autor.".

O ponto em que as duas curvas se separam marca o início do overfitting e dá a base para a parada antecipada.

Para perceber o overfitting antes que ele apareça em produção, é preciso medir o desempenho do modelo em dados que ele não viu durante o ajuste. Por isso o conjunto de dados rotulados é repartido em três partes com papéis distintos: um conjunto de treino, sobre o qual os parâmetros são ajustados; um conjunto de validação, usado para comparar modelos e escolher hiperparâmetros; e um conjunto de teste, reservado para uma estimativa final e honesta da generalização. A figura a seguir mostra como esses três conjuntos se relacionam ao longo do trabalho.

Figura 2. Ciclo treino, validação e teste.



Fonte: autor.

"Para cego ver":

Imagem de um diagrama intitulado "Figura 2. Ciclo treino, validação e teste". O esquema apresenta o fluxo de desenvolvimento e avaliação de um modelo de aprendizado de máquina, organizado em duas etapas principais.

Na parte superior, sob o título "1. Partição do conjunto de dados rotulados", há três blocos retangulares alinhados horizontalmente. O primeiro, maior, é denominado Treino e corresponde a aproximadamente 60% dos dados. Ao lado, há o bloco Validação, com aproximadamente 20% dos dados. O terceiro bloco é Teste, também com aproximadamente 20% dos dados.

Uma seta direciona para a segunda etapa, intitulada "2. Laço de ajuste e avaliação final". Nessa etapa, há três blocos conectados por setas. O primeiro bloco, Treinar, informa que o modelo ajusta seus parâmetros utilizando o conjunto de treino. Uma seta leva ao bloco Validar, onde os modelos são comparados e os hiperparâmetros são escolhidos com base no conjunto de validação. Entre os blocos de treino e validação há setas de ida e volta, representando um processo iterativo. Abaixo delas, aparece a observação: "iterar: repetir até o desempenho de validação estabilizar".

Após a validação, uma seta segue para o bloco Testar, acompanhado da indicação "uma vez". Sobre a seta está indicado que o modelo final selecionado é enviado para teste.

Na parte inferior da figura há uma caixa intitulada "Regra de ouro". O texto informa que o conjunto de teste serve para estimar a capacidade de generalização do modelo e deve ser utilizado apenas uma única vez. Caso seja usado para orientar ajustes, a estimativa torna-se excessivamente otimista devido ao vazamento de dados, deixando de representar adequadamente o desempenho em novos dados.

O diagrama utiliza caixas de cantos arredondados, setas e diferentes tonalidades de azul e cinza para organizar visualmente as etapas do processo.

O par treino-validação é iterado; o teste é usado uma só vez, ao final.

Resta saber como medir o erro, e aqui a escolha depende da tarefa. Em problemas de

classificação, a acurácia mede a fração de acertos no total, mas esconde desequilíbrios: num conjunto em que 99% dos casos são negativos, um modelo que sempre responde negativo tem 99% de acurácia e nenhuma utilidade. Por isso medem-se também a precisão, que conta quantas das previsões positivas estavam corretas, e o recall, que conta quantos dos casos positivos reais foram capturados, sintetizados pelo F1. Em problemas de regressão, a saída é um número, e o que se mede é o desvio entre o valor previsto e o real, com a escolha entre tratar todos os erros com peso igual ou penalizar mais os erros grandes. Os dois quadros a seguir reúnem as métricas mais usadas, com a definição em palavras de cada uma.

Quadro 2. Métricas de classificação.

Métrica	Para que serve	Fórmula verbal
Acurácia	Fração de acertos no total	Acertos sobre total
Precisão	Quanto das previsões positivas estava correto	Acertos positivos sobre previsões positivas
Recall	Quanto dos positivos reais foi capturado	Acertos positivos sobre positivos reais
F1	Síntese de precisão e recall	Média harmônica de precisão e recall
Matriz de confusão	Mapa de acertos e erros por categoria	Tabela de combinações real por previsto

"Para cego ver":
Quadro 2, intitulado "Métricas de classificação", apresenta as principais métricas utilizadas para avaliar modelos de classificação, explicando suas funções e respectivas fórmulas conceituais.
A primeira métrica apresentada é a Acurácia, que serve para indicar a fração de acertos no total, sendo calculada verbalmente como os acertos sobre o total.
Em seguida, a Precisão avalia quanto das previsões positivas estava correto, expressa pela fórmula de acertos positivos sobre previsões positivas.
O Recall mede quanto dos positivos reais foi capturado pelo modelo, tendo como fórmula verbal os acertos positivos sobre positivos reais.
A métrica F1 funciona como uma síntese de precisão e recall, sendo definida como a média harmônica de precisão e recall.
Por fim, a Matriz de confusão atua como um mapa de acertos e erros por categoria, estruturada como uma tabela de combinações do valor real por previsto.

Quadro 3 - Métricas de regressão.

Métrica	Para que serve	Fórmula verbal
Erro absoluto médio (MAE)	Tamanho médio do erro, com peso linear	média da diferença em valor absoluto entre previsto e real
Raiz do erro quadrático médio (RMSE)	Tamanho médio do erro, penalizando mais os erros grandes	raiz quadrada da média do quadrado da diferença entre previsto e real

Métrica	Para que serve	Fórmula verbal
Coeficiente de determinação (R^2)	Fração da variação dos dados que o modelo explica	1 indica explicação perfeita; 0 indica que o modelo não explica nada; pode ser negativo

"Para cego ver":

Tabela intitulada "Quadro 3. Métricas de regressão", organizada em três colunas: Métrica, Para que serve e Fórmula verbal.

A primeira linha apresenta o Erro Absoluto Médio (MAE), descrito como uma métrica que mede o tamanho médio do erro com peso linear. Sua fórmula verbal é a média da diferença, em valor absoluto, entre o valor previsto e o valor real.

A segunda linha apresenta a Raiz do Erro Quadrático Médio (RMSE), utilizada para medir o tamanho médio do erro, atribuindo maior penalização aos erros mais elevados. Sua fórmula verbal corresponde à raiz quadrada da média dos quadrados das diferenças entre os valores previstos e os reais.

A terceira linha apresenta o Coeficiente de Determinação (R^2), que indica a fração da variação dos dados explicada pelo modelo. Na interpretação apresentada, valor 1 representa explicação perfeita; valor 0 indica que o modelo não explica a variação dos dados; e o resultado pode assumir valores negativos.

A tabela possui cabeçalho em azul-claro e células com fundo branco e cinza-claro, separadas por linhas horizontais.

A escolha entre métricas depende do custo dos erros, não apenas do nome da tarefa. Em um sistema que filtra correio indesejado, marcar uma mensagem legítima como spam (um falso positivo) é mais doloroso para o usuário que deixar passar um spam (um falso negativo), e por isso a precisão tende a importar mais que o recall, porque cada falso positivo é uma mensagem importante perdida. Já em um exame de triagem para uma doença grave, deixar passar um caso positivo é incomparavelmente pior que pedir um exame de confirmação extra, e por isso o recall tende a importar mais. O F1 ajuda quando os dois lados doem de maneira parecida e não se quer privilegiar um.

Em regressão, a mesma lógica vale: o RMSE expõe mais os erros catastróficos porque eleva-os ao quadrado, o MAE trata todos com peso linear, e a escolha depende de quanto um erro grande custa frente a vários erros pequenos. Como regra prática, vale sempre olhar mais de uma métrica em conjunto: uma única métrica é fácil de otimizar e fácil de enganar.

Quando os dados são poucos para sustentar uma divisão fixa em três conjuntos, recorre-se à validação cruzada, que reaproveita os dados em múltiplas partições rotativas.

Outro fator que afeta o uso das métricas é o balanceamento das classes. Quando uma das categorias é muito mais frequente que a outra, digamos, 95% dos exemplos pertencem à classe negativa e 5% à positiva, a acurácia perde sentido como métrica isolada, e o modelo pode optar pela resposta majoritária sem aprender nada.

Estratégias práticas para essa situação incluem reportar precisão, recall e F1 em lugar da acurácia, sobreamostrar a classe rara durante o treinamento, subamostrar a classe abundante ou atribuir pesos diferentes aos erros de cada classe na função que se quer minimizar. Esse cuidado é tão importante em problemas industriais reais (falhas, fraudes, defeitos) que mereceu um exemplo dedicado a seguir, em que a raridade da classe positiva determina a métrica que importa.

Exemplo aplicado

Numa linha de manutenção preditiva, uma fábrica registra vibração, temperatura e corrente de motores elétricos e quer prever falhas antes que ocorram. Os dados são tabulares, com uma coluna que marca se houve falha nos dias seguintes. O problema é de classificação, e a métrica não pode ser só acurácia: falhas são raras, então recall e precisão importam mais, porque deixar passar uma falha custa uma parada não planejada, enquanto um alarme falso custa uma inspeção.

A disciplina de separar os conjuntos só funciona se for respeitada sem exceção, e é justamente aí que erros comuns aparecem.

Armadilha comum

Um erro frequente é medir o desempenho no próprio conjunto de treinamento e comemorar a acurácia alta. Ela quase sempre é otimista, porque o modelo já viu aquelas respostas. Erro relacionado e mais sutil é deixar informação do teste vazar para o ajuste, por exemplo normalizando todos os dados juntos antes de separá-los. A consequência é a mesma: a estimativa de desempenho fica inflada e o modelo decepciona em produção, quando os dados são de fato novos.

Com esses termos firmes, modelo e parâmetro, treinamento e inferência, generalização e overfitting, conjuntos e métricas, você tem o vocabulário mínimo para situar qualquer técnica. A próxima seção usa esse vocabulário para percorrer o arco histórico e arquitetural do campo.

0.2. Arco histórico e arquitetural

Esta seção pertence à camada conceitual e tem por objetivo que você reconheça o arco histórico-arquitetural do campo e saiba situar uma técnica específica entre quatro grandes famílias: ML clássico, Deep Learning, Foundation Models e sistemas agênticos. O propósito não é decorar datas, e sim ganhar um mapa que permita posicionar uma solução qualquer e antecipar suas exigências de dados, computação e validação.

A história começa em 1958, com o perceptron, um modelo capaz de aprender a separar duas classes ajustando pesos sobre entradas numéricas. Era uma única camada de decisão, limitada a problemas linearmente separáveis, e essa limitação esfriou o interesse por quase duas décadas.

A retomada veio em 1986, quando a retropropagação tornou prático treinar redes com várias camadas, abrindo caminho para modelos mais expressivos. Ainda assim, durante os anos 1990 e 2000 o que dominava as aplicações eram técnicas hoje agrupadas sob o rótulo de ML clássico: árvores de decisão, florestas aleatórias, máquinas de vetores

de suporte e métodos de regressão. São técnicas que funcionam bem com dados tabulares e quantidades moderadas de exemplos, exigem pouca computação e continuam sendo a escolha certa para muitos problemas industriais.

Entre o perceptron de uma camada e a explosão das redes profundas em 2012 passaram-se mais de cinco décadas, e a maior parte desse intervalo foi de avanços contidos. Não faltou inteligência teórica: a retropropagação existia desde 1986, e os princípios das redes convolucionais e das LSTM já estavam descritos nos anos 1990. O que faltava eram três insumos práticos. Primeiro, dados em volume: treinar uma rede profunda exige milhões de exemplos, e essa escala só ficou disponível com a internet e com bases públicas como ImageNet, anotada manualmente entre 2009 e 2010.

Segundo, computação capaz de absorver esse volume: as placas gráficas, ou GPUs, originalmente desenhadas para jogos, ofereciam o tipo certo de paralelismo, e por volta de 2010 ficaram acessíveis a laboratórios acadêmicos. Terceiro, um conjunto de truques de engenharia, entre eles funções de ativação como ReLU, dropout e normalização de batch, que tornaram o treinamento de redes profundas estável de fato, e não apenas no papel. O marco visível dessa convergência foi a AlexNet, em 2012: uma rede convolucional grande, treinada em GPU sobre o ImageNet, cuja vitória num torneio de visão computacional anunciou que a era do Deep Learning estava começando.

Apesar da atenção que o Deep Learning concentra hoje, o ML clássico continua sendo a escolha apropriada para boa parte dos problemas industriais. Três condições típicas o favorecem. Quando os dados são tabulares e em volume moderado, da ordem de milhares a centenas de milhares de linhas, uma floresta aleatória ou um algoritmo de gradient boosting frequentemente bate uma rede profunda em precisão, com uma fração do custo computacional.

Quando o orçamento de inferência é apertado, seja por restrição de hardware ou por exigência de latência baixa, modelos clássicos com poucos parâmetros respondem em microssegundos sem hardware especializado. Quando a auditabilidade importa, em decisões de crédito, saúde ou conformidade regulatória, métodos clássicos costumam ser mais fáceis de explicar e justificar, porque suas regras internas podem ser inspecionadas e narradas em termos humanos. Em todos esses casos, partir direto para a rede mais nova é trocar uma solução adequada por uma cara, lenta e opaca.

GLOSSÁRIO

Foundation Model

Modelo de grande escala, treinado em volume amplo de dados gerais, que serve de base para muitas tarefas diferentes mediante adaptação leve, em vez de ser treinado do zero para cada tarefa. Modelos de linguagem grandes são o exemplo mais conhecido. O termo foi popularizado por um relatório de Stanford em 2021.

A virada para o que se chama de Deep Learning aconteceu por volta de 2012, quando redes neurais profundas, treinadas em grandes volumes de dados e aceleradas por placas gráficas, passaram a superar de forma marcante as técnicas anteriores em visão computacional. Nessa época, três famílias de arquitetura organizaram:

- **Perceptron multicamadas (multilayer perceptron, ou MLP):** Rede totalmente conectada, é a mais geral, pois cada neurônio de uma camada conecta-se a todos da camada seguinte, sem assumir nada sobre a estrutura da entrada. Funciona razoavelmente para dados tabulares e tarefas simples, mas perde para alternativas especializadas quando os dados têm estrutura espacial ou sequencial.
- **Rede convolucional, ou CNN:** explora a estrutura espacial das imagens aplicando o mesmo filtro a regiões locais e compondo o resultado em camadas; com isso, captura padrões visuais como bordas, texturas e formas sem que o engenheiro precise nomeá-los um a um.
- **Rede recorrente, ou RNN:** Com destaque para a variante LSTM, processa dados em sequência mantendo um estado interno que carrega informação de um passo para o seguinte, o que a torna adequada a texto, áudio e séries temporais.

Estas arquiteturas cobrem boa parte das aplicações herdadas e ainda em uso atualmente. Observe que o traço comum entre elas é o aprendizado de representações em camadas: em vez de o engenheiro definir manualmente quais atributos importam, a rede descobre atributos cada vez mais abstratos a partir dos dados brutos. cada vez mais abstratos a partir dos dados brutos.

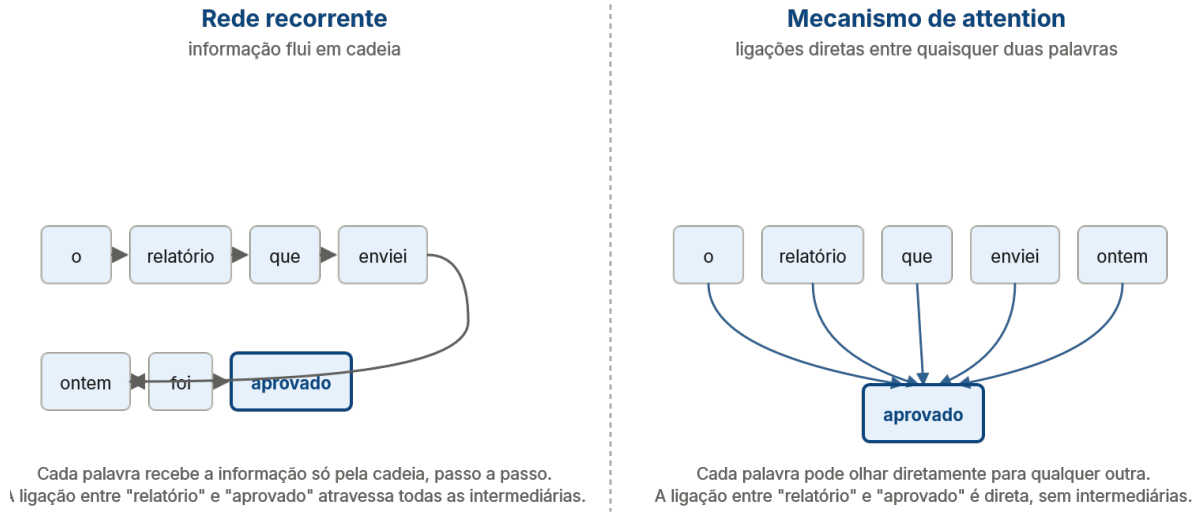
Reconhecer essas três famílias ajuda a ler especificações de projeto e escolher o ponto de partida natural conforme o tipo de dado.

O passo seguinte foi a introdução do mecanismo de attention e da arquitetura transformer, em 2017, que processa uma sequência inteira em paralelo e modela relações de

longa distância entre seus elementos melhor que as redes recorrentes.

> **Exemplo**
>
> A diferença fica mais fácil de ver com um exemplo. Numa frase como “o relatório que enviei ontem foi aprovado”, uma rede recorrente entrega informação palavra a palavra, em cadeia, e por isso enfraquece quando precisa ligar “aprovado” a “relatório”. O mecanismo de attention permite que cada palavra olhe diretamente para qualquer outra, sem passar por todas as intermediárias, e essa ligação direta é o que sustenta os modelos de hoje.

Figura 3. Rede recorrente e mecanismos de attention.



Fonte: autor.

“Para cego ver”:

Figura intitulada “Figura 3. Rede recorrente e mecanismos de attention”. A imagem compara duas abordagens de processamento de linguagem natural, apresentadas lado a lado.

À esquerda está a Rede recorrente, acompanhada da descrição “informação flui em cadeia”. As palavras da frase “o relatório que enviei ontem foi aprovado” aparecem conectadas em sequência por setas, formando um fluxo linear. Para relacionar as palavras “relatório” e “aprovado”, a informação precisa percorrer várias etapas intermediárias da frase. A legenda destaca que cada palavra recebe e transmite informações passo a passo, tornando a ligação entre termos distantes mais longa e indireta.

À direita está o Mecanismo de attention, descrito como “ligações diretas entre quaisquer duas palavras”. As palavras da mesma frase aparecem alinhadas horizontalmente, enquanto várias setas curvas convergem diretamente para a palavra “aprovado”, destacada por uma borda azul. Nesse modelo, palavras como “relatório”, “que”, “enviei”, “o” e “ontem” podem se conectar diretamente ao termo relevante, sem depender de etapas intermediárias. A legenda informa que cada palavra pode olhar diretamente para qualquer outra, permitindo uma ligação imediata entre “relatório” e “aprovado”.

Visualmente, a figura enfatiza a diferença entre um fluxo sequencial de informações, na rede recorrente, e conexões diretas entre palavras, no mecanismo de attention.

Como a informação flui em uma rede recorrente (à esquerda) e no mecanismo de attention (à direita), sobre uma mesma frase de exemplo.

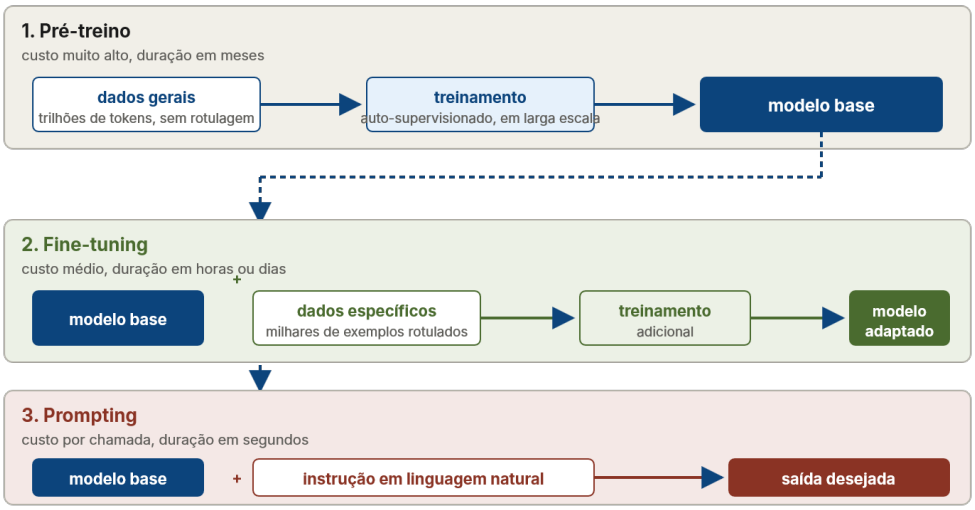
Sobre essa arquitetura cresceram os **Foundation Models:** modelos treinados em escala muito maior, em dados gerais, capazes de servir a muitas tarefas com pouca adaptação. Entre 2018 e 2020, modelos como BERT e as primeiras gerações de GPT mostraram esse padrão, e desde então ele define a fronteira de capacidade em linguagem e, cada vez mais, em outras modalidades.

Vale entender o que torna um Foundation Model “fundacional”. Três traços convivem: a escala (centenas de milhões a centenas de bilhões de parâmetros, treinados em dezenas de bilhões de exemplos); a generalidade do dado de treino (não se treina o modelo para uma tarefa específica, mas em corpus amplo que cobre muitos domínios); e a

possibilidade de adaptação leve para tarefas específicas, sem repetir o pré-treino. Essa adaptação tem duas formas principais. O fine-tuning toma o modelo pré-treinado e continua o treinamento sobre dados específicos da tarefa-alvo, ajustando todos os parâmetros ou um subconjunto deles; variantes recentes, como LoRA, fazem isso de forma muito mais barata, ajustando matrizes pequenas que se somam aos pesos originais. O prompting dispensa qualquer treinamento extra: o modelo recebe uma instrução em linguagem natural, possivelmente acompanhada de exemplos, e produz a saída desejada apenas com base no contexto.

A figura a seguir compara esses três caminhos de uso:

Figura 4. Três caminhos de uso de um Foundation Model: pré-treino do zero (custoso e raro), fine-tuning para adaptação (intermediário) e prompting para uso direto (barato e imediato).



Fonte: autor.

"Para cego ver":

Infográfico intitulado Figura 4, "Três caminhos de uso de um Foundation Model", dividido em três faixas horizontais coloridas que representam diferentes formas de utilizar um modelo de inteligência artificial.

Na parte superior, está o Pré-treino. O processo começa com uma grande quantidade de dados gerais, descritos como trilhões de tokens sem rotulação. Esses dados seguem para uma etapa de treinamento em larga escala, baseada em auto-supervisão, resultando em um modelo base. O texto informa que essa etapa possui custo muito alto e duração de meses.

Na parte central, está o Fine-tuning. O fluxo parte do modelo base, que recebe dados específicos compostos por milhares de exemplos rotulados. Esses dados passam por um treinamento adicional, gerando um modelo adaptado para uma tarefa ou contexto específico. A legenda informa que essa abordagem possui custo médio e duração de horas ou dias.

Na parte inferior, está o Prompting. O processo utiliza diretamente o modelo base, sem novo treinamento. Uma instrução em linguagem natural é fornecida ao modelo, que gera a saída desejada. O texto destaca que esse método tem custo por chamada e produz resultados em segundos.

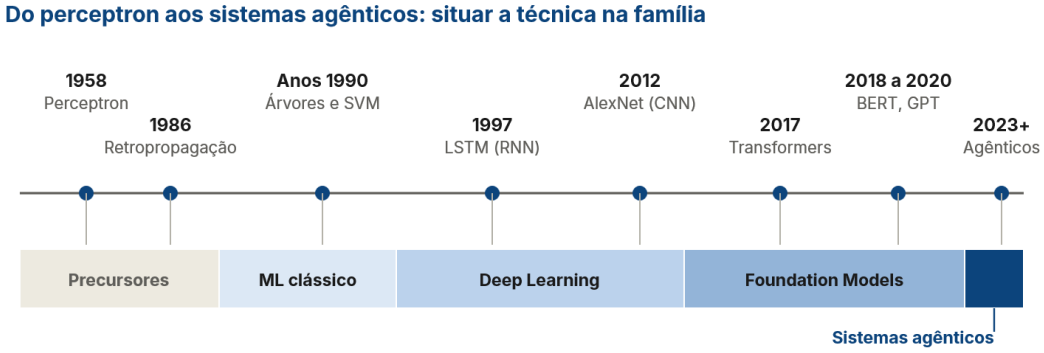
Setas conectam as etapas de cada fluxo, mostrando a progressão entre dados, treinamento e resultado. Uma seta tracejada liga o pré-treino às demais abordagens, indicando que tanto o fine-tuning quanto o prompting dependem de um modelo base previamente treinado. O diagrama compara visualmente os três caminhos, evidenciando diferenças de custo, tempo e nível de adaptação do modelo.

A partir de 2023 ganham tração os sistemas agênticos, que envolvem um modelo de linguagem em um laço de decisão com ferramentas e memória. A motivação é prática: um Foundation Model isolado responde, com excelência, a perguntas em uma única passagem, mas não realiza ações no mundo, não consulta sistemas externos para verificar fatos e não persegue objetivos de várias etapas.

O paradigma agêntico adiciona três componentes ao redor do modelo. Um conjunto de ferramentas que o modelo pode acionar (busca na web, execução de código, consulta a um banco de dados, envio de e-mail). Uma memória que persiste entre passos, para que o sistema lembre o que já fez e o que descobriu. Um laço de decisão que, a cada passo, escolhe a próxima ação a partir do estado atual e do objetivo declarado. O módulo M2 trata desses três componentes em detalhe.

A figura a seguir organiza esse arco em famílias:

Figura 5. Arco histórico-arquitetural agrupado por família.



Fonte: autor.

"Para cego ver":

Figura 5, intitulada "Arco histórico-arquitetural agrupado por família". A imagem apresenta uma linha do tempo horizontal que mostra a evolução das principais abordagens de Inteligência Artificial e Aprendizado de Máquina, desde os primeiros modelos até os sistemas agênticos atuais.

Na parte superior, o subtítulo informa: "Do perceptron aos sistemas agênticos: situar a técnica na família".

Ao longo da linha do tempo, são destacados os seguintes marcos:

1958: Perceptron.

1986: Retropropagação.

Anos 1990: Árvores de decisão e máquinas de vetores de suporte (SVM).

1997: LSTM, uma arquitetura de redes neurais recorrentes.

2012: AlexNet, modelo de rede neural convolucional.

2017: Transformers.

2018 a 2020: BERT e GPT.

2023 em diante: Sistemas agênticos.

Abaixo da linha do tempo, as técnicas são agrupadas em cinco grandes famílias representadas por blocos coloridos em tons de azul:

Precursores, abrangendo os primeiros desenvolvimentos da área.

Machine Learning Clássico, associado às técnicas predominantes dos anos 1990.

Deep Learning, que inclui redes neurais profundas e arquiteturas como LSTM e AlexNet.

Foundation Models, representados pelos Transformers, BERT e GPT.

Sistemas Agênticos, destacando a geração mais recente de sistemas capazes de executar ações e tomar decisões de forma mais autônoma.

Na parte inferior, uma observação explica que cada técnica pertence a uma família e que, ao escolher uma solução, é importante identificar a família tecnológica mais adequada ao problema, sem presumir que a mais recente seja necessariamente a melhor opção. Há ainda uma nota informando que a linha do tempo é esquemática, com posições aproximadas e sem escala exata.

Fonte informada na figura: autor.

As posições são aproximadas e não estão em escala.

O valor desse mapa está em ajudar a situar uma técnica, e situar bem evita uma confusão que custa caro em projetos.

Exemplo aplicado

Uma empresa de inspeção visual quer detectar defeitos em peças fotografadas na linha de produção. O problema é de visão sobre imagens, e a família natural é o Deep Learning, com uma rede convolucional. Já um setor de crédito da mesma empresa quer prever inadimplência a partir de uma planilha com histórico de pagamentos. Os dados são tabulares e em volume moderado; aqui uma técnica de ML clássico, como uma floresta aleatória, tende a render mais com menos custo. Mesma empresa, famílias diferentes, porque os dados e a tarefa diferem.

Armadilha comum

Uma armadilha recorrente é supor que a família mais recente é sempre a melhor, e tentar resolver com um Foundation Model um problema que uma árvore de decisão sobre dados tabulares resolveria com mais precisão, menor custo e mais facilidade de auditoria. A consequência é um sistema caro, lento e difícil de explicar, escolhido por moda e não por adequação. Situar a técnica na família certa começa pela natureza dos dados e da tarefa, não pela data de publicação do método.

Para aprofundar

Para um panorama atualizado de desempenho e adoção das famílias arquiteturais, veja STANFORD HAI, Artificial Intelligence Index Report 2025, capítulos 1 e 2, em hai.stanford.edu/ai-index. Para o conceito de Foundation Model, BOMMASANI, R. et al., On the Opportunities and Risks of Foundation Models, em arxiv.org/abs/2108.07258.

Vale registrar, antes de fechar este arco, que em ambientes de PD&I as quatro famílias coexistem em vez de se substituírem. Um mesmo projeto pode envolver uma floresta aleatória para uma previsão tabular auxiliar, uma rede convolucional para processar imagens de inspeção, um Foundation Model para extrair informação de documentos técnicos em linguagem natural e um pequeno sistema agêntico para orquestrar a sequência. A pergunta operacional não é “qual família é a melhor”, e sim “qual família é a mais adequada a esta parte específica do problema, dadas as restrições de dado, custo, latência e auditabilidade”.

Esse tipo de decisão por componente é o que distingue arquiteturas robustas em produção de demonstrações que funcionam só em laboratório. Os módulos seguintes oferecem as lentes para fazer essas escolhas com critério, percorrendo cada família em profundidade conforme o tema de cada lente exigir.

Com o arco em mãos, falta um segundo eixo de organização, complementar à arquitetura: como o modelo aprende e o que ele produz. É o tema da próxima seção.

0.3. Taxonomia de paradigmas e tarefas

Esta seção pertence à camada conceitual e acrescenta um elemento de reconhecimento aplicado: o objetivo é que você classifique um problema dado segundo o paradigma de aprendizagem e segundo a tarefa, fechando a competência M0.C1. A distinção entre esses dois eixos é a ideia mais importante da seção, porque eles são independentes e costumam ser confundidos.

O paradigma de aprendizagem descreve a partir de que tipo de sinal o modelo aprende. No aprendizado supervisionado, cada exemplo de treinamento vem acompanhado da

resposta correta, o rótulo, e o modelo aprende a reproduzir essa associação; é o caso de prever falhas a partir de exemplos rotulados como falha ou não falha.

> **No aprendizado não supervisionado, não há rótulos, e o modelo busca estrutura nos próprios dados, como grupos de clientes com comportamento parecido. No aprendizado por reforço, um agente aprende por tentativa e erro a partir de recompensas, ajustando uma política de ação para maximizar o retorno ao longo do tempo, típico de problemas de controle.**

Cada paradigma fica mais claro com um exemplo concreto.

No **supervisionado**, uma fábrica registra leituras de sensores de motores e marca, com base no histórico, quais leituras ocorreram pouco antes de uma falha; cada exemplo fica com o rótulo “falha” ou “normal” e o modelo aprende a prever esse rótulo a partir das leituras de um motor novo. A métrica natural envolve precisão e recall sobre falhas.

No **não supervisionado**, a mesma fábrica quer entender se seus motores se separam em grupos de comportamento parecido, sem definir os grupos de antemão; um algoritmo de clustering analisa as leituras e propõe a divisão, e a avaliação passa a depender de quanto a divisão é coerente e útil para inspeção humana.

No **aprendizado por reforço**, a fábrica quer otimizar a sequência de manutenções para minimizar paradas; um agente decide entre “esperar” e “manutenção preventiva” a cada turno e aprende, ao longo do tempo, qual sequência de decisões rende menos paradas, com base nas paradas que cada decisão causou ou evitou.

Os três paradigmas atacam realidades distintas e exigem investimentos diferentes em rotulagem, instrumentação e tempo de aprendizado.

Vale registrar o vocabulário específico do aprendizado por reforço, que difere bastante do supervisionado e aparece em qualquer projeto de controle. O agente atua sobre um ambiente; em cada passo observa um estado e escolhe uma ação a partir de uma política, recebendo de volta uma recompensa (positiva ou negativa) e o novo estado. A política é o que se aprende, e o objetivo é maximizar a soma das recompensas ao longo do tempo, e não a recompensa de cada passo isolado. Isso introduz um dilema central: o agente deve explorar ações pouco testadas, que podem render mais no longo prazo, ou explotar as que já se mostraram boas? O equilíbrio entre exploração e explotação é decidido por hiperparâmetros e estratégias que o módulo M2 trata em detalhe.

Entre o supervisionado e o não supervisionado situa-se um paradigma que sustenta

boa parte dos modelos recentes e merece atenção, porque costuma ser uma lacuna de vocabulário.

GLOSSÁRIO

Glossário – aprendizado auto-supervisionado

Paradigma em que os rótulos são gerados a partir dos próprios dados, sem anotação humana. Um modelo de linguagem aprende prevendo a próxima palavra de textos existentes: o texto é, ao mesmo tempo, entrada e resposta. É o que viabiliza treinar Foundation Models em grande escala, já que dispensa rotulagem manual de cada exemplo.

Vale entender por que o auto-supervisionado virou o motor dos modelos de linguagem grandes.

O problema clássico do supervisionado é que rotular dados custa caro e em volume; treinar um modelo competente exige milhões de exemplos rotulados por humanos, o que limita os domínios em que ele é viável. Texto, no entanto, traz dentro de si um sinal de treino gratuito: dada uma sequência de palavras, a tarefa de prever a próxima é bem definida e pode ser checada automaticamente contra a sequência original. Isso vale para qualquer texto disponível na internet, em bibliotecas digitais e em código aberto, sem nenhuma anotação adicional.

O mesmo princípio se estende a outras modalidades: prever o pedaço escondido de uma imagem, prever o próximo quadro de um vídeo, prever o áudio mascarado de uma gravação. A capacidade que esses modelos desenvolvem como subproduto desse exercício de previsão é o que se chama de representação aprendida, base para todas as adaptações posteriores via fine-tuning ou prompting.

O segundo eixo é a tarefa, que descreve o que o modelo produz, independentemente de como aprendeu. Classificação atribui uma entrada a uma categoria entre opções discretas. Regressão produz um número contínuo. Clustering agrupa exemplos por semelhança sem categorias predefinidas. Geração produz um artefato novo, como texto, imagem ou dado sintético. Recuperação encontra, num acervo, os itens mais relevantes para uma consulta. Controle escolhe ações que afetam um ambiente ao longo do tempo. Reconhecer a tarefa é o primeiro passo prático ao ler uma especificação, porque a tarefa determina a métrica de avaliação e restringe as famílias de técnica adequadas.

Para ancorar essas seis tarefas em casos concretos, o quadro a seguir reúne um exemplo industrial típico para cada combinação relevante de paradigma e tarefa, no contexto de problemas de PD&I.

Quadro 4. Exemplo industrial típico para cada combinação relevante de paradigma e tarefa.

Paradigma	Tarefa	Exemplo industrial
Supervisionado	Classificação	Detecção de defeitos em peças por imagem rotulada
Supervisionado	Regressão	Previsão de demanda de energia a partir de variáveis climáticas
Não supervisionado	Clustering	Segmentação de motores por padrão de operação
Auto-supervisionado	Geração	Modelo de linguagem aprendendo a prever o próximo token
Auto-supervisionado	Recuperação	Busca semântica em base de manuais técnicos
Por reforço	Controle	Otimização de scheduling em linha de produção

"Para cego ver":

Tabela intitulada "Quadro 4.

Exemplo industrial típico para cada combinação relevante de paradigma e tarefa". A tabela possui três colunas: Paradigma, Tarefa e Exemplo industrial.

Na primeira linha, o paradigma Supervisionado está associado à tarefa de Classificação, com o exemplo industrial de detecção de defeitos em peças por meio de imagens rotuladas.

Nas linhas seguintes, são apresentados outros exemplos:

Supervisionado | Regressão | Previsão de demanda de energia a partir de variáveis climáticas.

Não supervisionado | Clustering | Segmentação de motores com base em padrões de operação.

Auto-supervisionado | Geração | Modelo de linguagem aprendendo a prever o próximo token.

Auto-supervisionado | Recuperação | Busca semântica em bases de manuais técnicos.

Por reforço | Controle | Otimização de agendamento (scheduling) em linhas de produção.

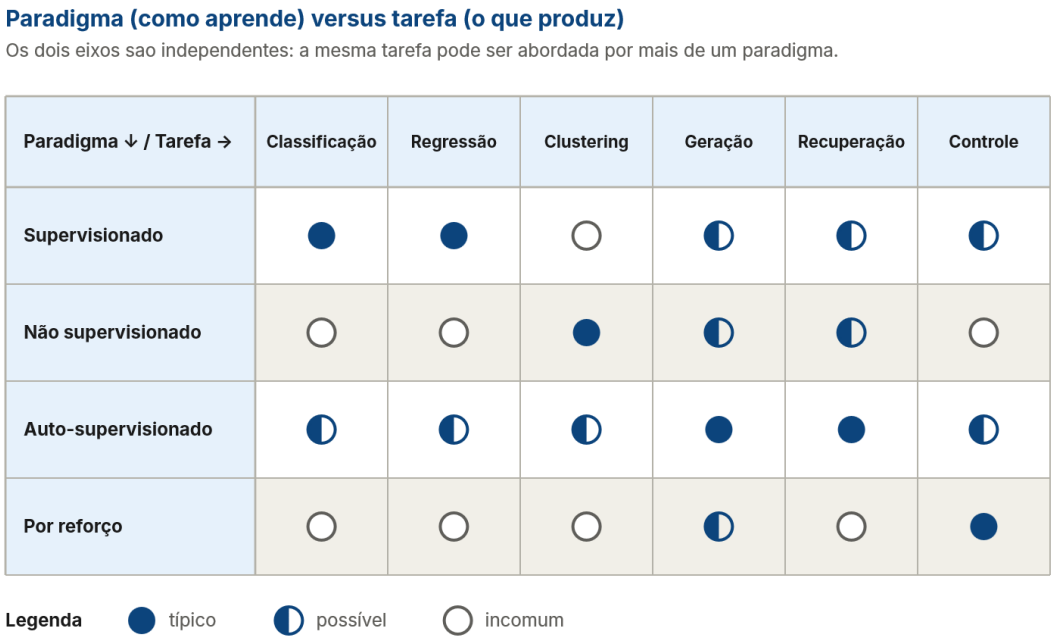
Visualmente, a tabela utiliza cabeçalho em azul-claro e linhas alternadas em tons claros. O quadro relaciona diferentes paradigmas de aprendizado de máquina às tarefas mais comuns e seus respectivos exemplos de aplicação industrial.

Olhando o quadro, fica clara a ortogonalidade dos dois eixos sobre o mesmo conjunto de dados. A fábrica que faz classificação supervisionada de defeitos por imagem pode, sobre as mesmas imagens, fazer clustering não supervisionado para identificar tipos de defeito não previstos no rótulo original; o paradigma muda, a tarefa muda, mas os dados são os mesmos. Reconhecer as duas dimensões separadamente é o que permite combinar abordagens em um mesmo projeto sem confundir o que cada uma faz e sem subestimar requisitos. A matriz a seguir formaliza essa ortogonalidade cruzando os quatro paradigmas com as seis tarefas.

A independência dos dois eixos fica clara quando se percebe que a mesma tarefa admite paradigmas diferentes e o mesmo paradigma serve a tarefas diferentes.

A figura a seguir cruza os quatro paradigmas com seis tarefas e marca, para cada combinação, se ela é típica, possível ou incomum.

Figura 6. Paradigmas de aprendizagem por tarefa.



Paradigma (como aprende) versus tarefa (o que produz)

Os dois eixos sao independentes: a mesma tarefa pode ser abordada por mais de um paradigma.

Paradigma ↓ / Tarefa →	Classificação	Regressão	Clustering	Geração	Recuperação	Controle
Supervisionado	●	●	○	◐	◐	◐
Não supervisionado	○	○	●	◐	◐	○
Auto-supervisionado	◐	◐	◐	●	●	◐
Por reforço	○	○	○	◐	○	●

Legenda ● típico ◐ possível ○ incomum

Fonte: autor.

"Para cego ver":

Tabela 6, intitulada "Paradigmas de aprendizagem por tarefa"

Paradigma (como aprende) versus tarefa (o que produz).

Logo abaixo do título, há uma observação informando que os dois eixos são independentes e que a mesma tarefa pode ser abordada por mais de um paradigma de aprendizado.

A tabela cruza quatro paradigmas de aprendizado com seis tipos de tarefas. Os paradigmas aparecem nas linhas: Supervisionado, Não supervisionado, Auto-supervisionado e Por reforço. As tarefas aparecem nas colunas: Classificação, Regressão, Clustering, Geração, Recuperação e Controle.

Os relacionamentos são representados por símbolos circulares, conforme a legenda:

Círculo azul preenchido: típico.

Círculo meio preenchido: possível.

Círculo vazio: incomum.

Na linha de aprendizado supervisionado, classificação e regressão aparecem como usos típicos; geração, recuperação e controle aparecem como possíveis; clustering aparece como incomum.

Na linha de aprendizado não supervisionado, clustering é típico; geração e recuperação são possíveis; classificação, regressão e controle são incomuns.

Na linha de aprendizado auto-supervisionado, geração e recuperação são típicas; classificação, regressão, clustering e controle aparecem como possíveis.

Na linha de aprendizado por reforço, controle é típico; geração é possível; classificação, regressão, clustering e recuperação são incomuns.

Visualmente, a tabela utiliza tons claros de azul e cinza, com linhas e colunas bem delimitadas. O objetivo é demonstrar que um mesmo tipo de tarefa pode ser realizado por diferentes paradigmas de aprendizado, embora algumas combinações sejam mais comuns do que outras.

Fonte: autor.

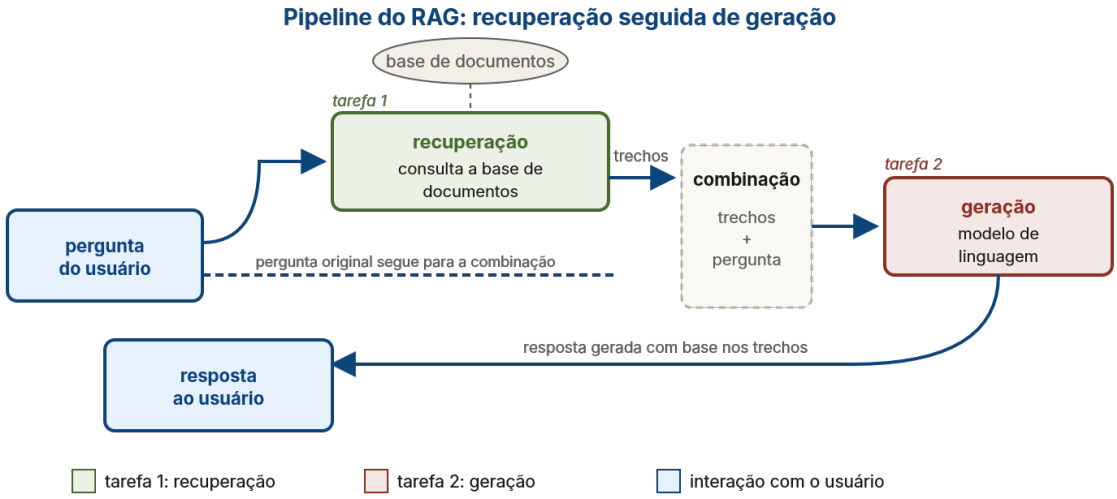
Os dois eixos são independentes; a mesma tarefa pode ser abordada por mais de um paradigma.

Ler a matriz por colunas é instrutivo. A geração, por exemplo, aparece como típica do auto-supervisionado, que treina modelos de linguagem prevendo o próximo elemento de uma sequência, mas também como possível no não supervisionado, com modelos que aprendem a distribuição dos dados, e no aprendizado por reforço, quando uma etapa de ajuste por recompensa refina um modelo gerador. Isso mostra que nomear a tarefa, geração, não basta para fixar a técnica; é preciso o segundo eixo. A leitura por linhas é igualmente útil: o aprendizado por reforço concentra-se em controle e raramente é a escolha para classificação tabular, o que ajuda a descartar abordagens inadequadas logo no início.

Sistemas reais raramente se resumem a uma única tarefa: o que se vê em produção é a composição de várias. Um exemplo cada vez mais comum é o RAG (retrieval-augmen-

ted generation), que combina recuperação e geração em um pipeline único. Dada uma pergunta do usuário, o sistema primeiro recupera os trechos mais relevantes de um acervo de documentos (tarefa de recuperação), e em seguida usa um modelo de linguagem para gerar a resposta com base nesses trechos (tarefa de geração). Outra composição frequente é o ranking sobre classificação: um modelo classifica candidatos por categoria e outro ordena por relevância. Reconhecer essas composições é importante porque cada etapa tem métricas, riscos e necessidades de dados próprios, e cobrar do sistema inteiro o mesmo critério que se cobraria de uma tarefa isolada leva a diagnósticos errados quando algo dá errado.

Figura 7. Pipeline do RAG (retrieval-augmented generation), exemplo de composição de duas tarefas: recuperação e geração.



Fonte: autor.

"Para cego ver":

Figura intitulada "Figura 7. Pipeline do RAG (retrieval-augmented generation), exemplo de composição de duas tarefas: recuperação e geração". A imagem apresenta um diagrama de fluxo que explica como funciona um sistema RAG, combinando a recuperação de informações em uma base de documentos com a geração de respostas por um modelo de linguagem.

No centro da figura há o subtítulo "Pipeline do RAG: recuperação seguida de geração".

O fluxo inicia à esquerda com um bloco azul-claro denominado "pergunta do usuário". A partir dele, uma seta azul segue para a etapa de recuperação. Paralelamente, uma linha tracejada indica que a pergunta original também será encaminhada diretamente para uma etapa posterior de combinação.

A primeira etapa, destacada em verde-claro, é a "recuperação", descrita como "consulta à base de documentos". Acima desse bloco há um elemento representando a base de documentos, conectado à recuperação por uma linha tracejada vertical. Após a consulta, a recuperação retorna trechos relevantes de informação.

Esses trechos seguem por uma seta identificada como "trechos" até um bloco de bordas tracejadas denominado "combinação". Dentro desse bloco ocorre a junção entre os trechos recuperados e a pergunta original, representada pelo texto "trechos + pergunta".

Em seguida, uma seta conduz o conteúdo combinado para a segunda etapa, destacada em vermelho-claro, chamada "geração", descrita como "modelo de linguagem". Nessa fase, o modelo utiliza os trechos recuperados como contexto para produzir uma resposta.

Por fim, uma seta azul curva sai da etapa de geração e retorna para um bloco azul denominado "resposta ao usuário". Sobre essa seta está a descrição "resposta gerada com base nos trechos", indicando que a resposta final foi construída utilizando as informações recuperadas da base de documentos.

Visualmente, a figura diferencia as etapas por cores: azul para interação com o usuário, verde para recuperação de informações e vermelho para geração de texto. O diagrama demonstra como o RAG combina busca de conteúdo e geração de linguagem para produzir respostas fundamentadas em documentos previamente consultados.

Fonte: autor.

Exemplo aplicado

Uma indústria têxtil quer, a partir de fotos de tecidos, separar peças com e sem defeito e, num segundo momento, agrupar os defeitos por tipo para investigar causas. A primeira parte é uma tarefa de classificação com paradigma supervisionado, pois há fotos rotuladas como defeito ou não. A segunda é uma tarefa de clustering com paradigma não supervisionado, pois os tipos de defeito não foram definidos de antemão. É o mesmo acervo de imagens servindo a duas combinações distintas de paradigma e tarefa.

A confusão mais comum nesta etapa não é técnica, mas de vocabulário, e propaga erros de planejamento.

Armadilha comum

A armadilha mais frequente é tratar paradigma e tarefa como se fossem a mesma coisa, dizendo, por exemplo, que um projeto é de classificação para descrever como o modelo aprende, ou chamando o aprendizado supervisionado de tarefa. A consequência prática é subestimar requisitos: ao descrever a tarefa sem nomear o paradigma, esquece-se de perguntar se existem rótulos, quem vai produzi-los e a que custo. Muitos projetos travam não por falta de modelo, mas por falta de dados rotulados que ninguém previu.

Para classificar um problema novo na prática, vale aplicar uma sequência curta de perguntas.

Primeiro: há rótulos disponíveis para os exemplos, ou existe orçamento e capacidade de produzi-los? Se sim, e o objetivo é prever o rótulo, o paradigma é supervisionado; se não, o caminho passa pelo não supervisionado ou pelo auto-supervisionado, conforme o sinal de treino que se consiga extrair dos próprios dados.

Segundo: o sistema age sobre um ambiente e recebe consequências entre passos, ou apenas responde a entradas isoladas? Se age, o paradigma é por reforço.

Terceiro: o que o sistema produz, em cada chamada? Uma categoria (classificação), um número (regressão), um agrupamento sem rótulos prévios (clustering), um artefato novo (geração), itens recuperados de um acervo (recuperação) ou uma decisão de ação (controle)? A combinação das respostas posiciona o problema na matriz, antecipa quais técnicas e métricas tendem a ser adequadas e expõe, com clareza, onde estarão os principais riscos de execução.

SAIBA MAIS

Para uma introdução cuidadosa a paradigmas, tarefas e avaliação, com o vocabulário formal de aprendizado de máquina, veja MURPHY, K. P., Probabilistic Machine Learning: An Introduction, MIT Press, 2022, capítulo 1, disponível em probml.github.io/pml-book. Para fundamentos de agentes e paradigmas, POOLE, D. L.; MACKWORTH, A. K., Artificial Intelligence: Foundations of Computational Agents, 3. ed., capítulos 1 e 2, em artint.info/3e.

Conclusão

As três seções deste texto-base cobriram conjuntamente o repertório mínimo de leitura técnica que sustenta o restante do curso: o vocabulário fundamental de modelagem e avaliação (modelo, treinamento, generalização, métricas, conjuntos de dados), o arco histórico-arquitetural das principais famílias técnicas (do perceptron ao Foundation Model, passando por ML clássico, Deep Learning e arquitetura transformer), e a taxonomia de paradigmas e tarefas como dois eixos independentes para classificar um problema. Cada

um desses três blocos foi tratado em alto nível, sem implementação em ferramenta nem matemática avançada, com foco na nomeação correta e no reconhecimento de famílias.

O ganho prático desse vocabulário comum vai além da precisão conceitual. Numa equipe com origens disciplinares variadas, nomear o paradigma de aprendizagem em jogo e a tarefa que o sistema executa torna explícito o que cada parte interessada está pressupondo sobre dados, restrições e métricas. Saber dizer, sem hesitação, “este é um problema supervisionado de classificação com classes desequilibradas” comunica em uma frase aquilo que, sem a taxonomia, exigiria várias rodadas de esclarecimento. Exigências de execução escondidas nas escolhas técnicas, como a produção de rótulos, o sinal de treino disponível ou o ambiente de inferência, sobem à superfície no momento certo, que é o de delimitar o problema, e não o de escolher a ferramenta.

A próxima peça do módulo apresenta um segundo recurso de organização, distinto e complementar à taxonomia: as cinco lentes transversais. Enquanto a taxonomia classifica o que um sistema é, as lentes determinam por quais ângulos ele deve ser lido. Cada uma delas se desenvolve em profundidade num módulo seguinte (M1 a M5), e juntas constituem o quadro por meio do qual qualquer problema técnico encontrado durante o programa poderá ser examinado.

REFERÊNCIAS

MURPHY, K. P. Probabilistic Machine Learning: An Introduction. MIT Press, 2022. Capítulo 1. Disponível gratuitamente em https://probml.github.io/pml-book.

POOLE, D. L.; MACKWORTH, A. K. Artificial Intelligence: Foundations of Computational Agents. 3. ed. Cambridge University Press, 2023. Capítulos 1 e 2. Disponível gratuitamente em https://artint.info/3e/html/ArtInt3e.html.

STANFORD HAI. Artificial Intelligence Index Report 2025. Stanford University, 2025. Capítulos 1 e 2 (Research and Development; Technical Performance). Disponível em https://hai.stanford.edu/ai-index.

Bibliografia complementar

BISHOP, C. M.; BISHOP, H. Deep Learning: Foundations and Concepts. Springer, 2024. PDF gratuito autorizado pelos autores em https://bishopbook.com.

BOMMASANI, R. et al. On the Opportunities and Risks of Foundation Models. CRFM Stanford, 2021. Preprint em https://arxiv.org/abs/2108.07258.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. Deep Learning. MIT Press, 2016. Capítulo 1. Disponível gratuitamente em https://www.deeplearningbook.org.

Industr.IA

Residência em TIC 49



Executor iRede

Executor Atlântico

Coordenador Softex

Financiador MCTI Futuro

Ministério da Ciência, Tecnologia e Inovação

Governo do Brasil — Do lado do povo brasileiro

Projeto apoiado pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248, de 23 de outubro de 1991.